



VLSI Lab

CSE 405

Experiment Number: 02

Design and Simulation of an 8-Bit Full Adder Using Verilog HDL in Xilinx ISE

Date: 27/09/2026

Submitted by:

Name : Md. Seeman Islam Seemanto
ID : 22201095
Section : C1
Semester : 4.2

Submitted to:

Md. Rasheduzzaman
Lecturer
Department of Computer Science and Engineering
University of Asia Pacific

Experiment Name

Design and Simulation of an 8-Bit Full Adder Using Verilog HDL (Ripple Carry Adder)

Objective

- Build an 8-bit ripple-carry adder by structurally wiring together eight instances of the 1-bit full_adder module.
- Reuse the 1-bit full_adder module from Experiment 02 as the core building block.
- Generate and read the RTL schematic of the 8-bit adder in Xilinx ISE, including the internal carry chain.
- Write Verilog testbenches — a fixed-vector version and a self-checking randomized version — to functionally verify the design.
- Hand-calculate the expected Sum and Cout for a representative set of 8-bit input combinations, including edge cases such as all-zero, all-one, and carry-overflow situations.
- Simulate the circuit in ISim and check the Sum and Cout waveforms against the calculated expectations.
- Confirm correct ripple-carry behaviour across all tested combinations, including 15 randomly generated vectors.

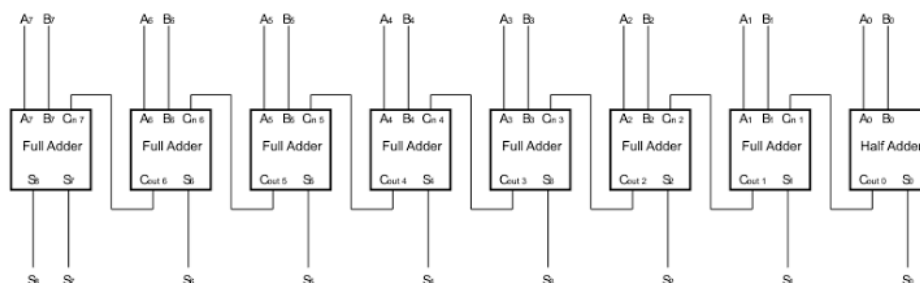
Hardware and Software Requirements

Tool / Component	Details
Xilinx ISE 14.7	The tool used to design, synthesize and verify the 8-bit full adder in Verilog.
Verilog HDL	The hardware description language used for the dataflow full adder and the structurally instantiated 8-bit ripple-carry adder.
ISim Simulator	Used to run the behavioral simulation and verify the 8-bit adder's functionality.
Artix-7 (xc7a100t-3csg324)	The target FPGA device chosen in Xilinx ISE for synthesis.
Computer / Virtual Machine	A Windows 7 VM inside VirtualBox, used to run Xilinx ISE 14.7 for the design and simulation work.

8-Bit Full Adder

An 8-bit full adder scales up the 1-bit full adder from Experiment 02 so it can add two 8-bit numbers, A(7:0) and B(7:0), together with an incoming carry Cin, and produce an 8-bit Sum(7:0) plus a final carry-out Cout. Since one full adder can only combine three single-bit inputs, eight of them are chained in a ripple-carry arrangement: the Cout of bit position i feeds directly into the Cin of position i+1, so the carry ripples from the least significant bit (LSB) up to the most significant bit (MSB).

This is structural reuse in action — assembling a wider, more capable circuit from a smaller module that has already been proven correct — which is a central idea in digital design. Because full adder was already exhaustively checked in Experiment 02 (all 8 input combinations matched the truth table), it can now be treated as a trusted black-box building block; the point of this experiment is to confirm that the interconnection — the carry chain — is wired correctly, not to re-derive the single-bit Boolean logic.



Boolean Expression

Each of the eight full adder instances, FA0 through FA7, computes its own 1-bit Sum and Cout using precisely the same two Boolean equations that were derived and verified for the 1-bit full adder.

$$\text{Sum}[i] = A[i] \oplus B[i] \oplus C[i]$$

$$\text{Cout}[i] = A[i].B[i] + B[i].C[i] + A[i].C[i]$$

where $C[0] = \text{Cin}$ (external) and $C[i] = \text{Cout}[i-1]$ for $i = 1$ to 7

and the final 8-bit adder's $\text{Cout} = \text{Cout}[7]$

In plain terms: $\text{Sum}[i]$ is 1 whenever an odd number of $A[i]$, $B[i]$ and $C[i]$ are 1 (the familiar XOR pattern), and $\text{Cout}[i]$ is 1 whenever two or more of $A[i]$, $B[i]$, $C[i]$ are 1 — meaning a carry has to be generated and passed on to the next, more significant bit position.

Worked substitution example — bit 0 of Test 6 ($A = 11111111$, $B = 00000001$, $\text{Cin} = 0$)

$A_0 = 1$, $B_0 = 1$, $C_0 = \text{Cin} = 0$ (as the least significant bit, it takes the external Cin directly, not a carry from another full adder):

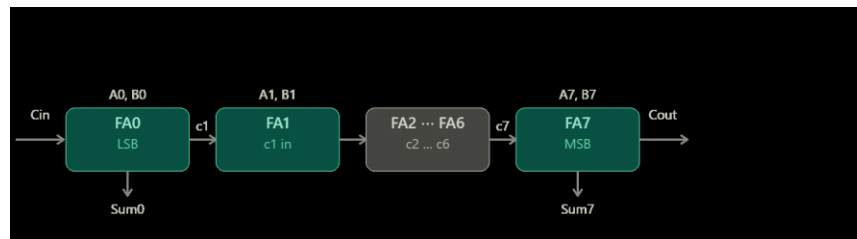
$$\text{Sum}[0] = A_0 \oplus B_0 \oplus C_0 = 1 \oplus 1 \oplus 0 = 0$$

$$\text{Cout}[0] = A_0.B_0 + B_0.C_0 + A_0.C_0 = (1.1) + (1.0) + (1.0) = 1 + 0 + 0 = 1$$

This $\text{Cout}[0] = 1$ becomes C_1 , the Cin feeding stage FA1 — exactly the handoff that drives the ripple-carry chain, worked out completely for all eight stages in the next section.

8-Bit Adder Is Built by Joining Eight 1-Bit Full Adders

A single full_adder can only combine three single-bit signals (A , B , Cin) into a single-bit Sum and Cout — on its own it has no way to add two 8-bit numbers. The 8-bit adder is assembled by placing eight full_adder copies side by side, one per bit position (FA0 at the LSB through FA7 at the MSB), and wiring each stage's Cout straight into the next stage's Cin :



FA0 is the only stage that is special: its Cin comes from the whole adder's external Cin input, since there's no bit "-1" to draw a carry from. Every other stage simply takes its Cin from the previous stage's Cout wire — in the Verilog these are the seven internal wires c_1 through c_7 . The 8-bit adder's overall external Cout is just FA7's Cout, with nothing further to ripple into.

Because each stage has to wait on the right carry-in before it can produce a valid carry-out, the carry effectively ripples from FA0 up to FA7 one stage at a time — the origin of the name ripple-carry adder. It also means the adder's true worst-case delay is not a single full-adder delay but as many as eight chained full-adder delays (the case where a carry has to propagate through every stage), which is exactly why the testbench allows #100 ns of settling time.

Fully worked example: Test 6 ($A = 11111111$, $B = 00000001$, $\text{Cin} = 0$) — the carry ripples through all 8 stages

This test case was picked deliberately because it produces the longest possible carry chain: adding 1 to 11111111 (255) forces every stage to generate and pass along a carry, all the way from FA0 to FA7. The bit-by-bit trace below shows what each of the eight full adders computes, and how each Cout feeds the next stage's Cin :

Stage	FA0	FA1	FA2	FA3	FA4	FA5	FA6	FA7
A[i]	1	1	1	1	1	1	1	1
B[i]	1	0	0	0	0	0	0	0
Cin (in)	0	1	1	1	1	1	1	1
Sum[i]	0	0	0	0	0	0	0	0
Cout (out)	1	1	1	1	1	1	1	1

Reading left to right: FA0 takes the external Cin = 0 and outputs Cout = 1 (because A0 = B0 = 1). That 1 becomes FA1's Cin; since A1 = 1 and B1 = 0, FA1 also outputs Cout = 1, and the same pattern repeats identically through FA7. Sum[i] comes out 0 for every stage i = 1..7, and Sum[0] is 0 as well, giving a final Sum of 00000000 — with the last carry, Cout[7] = 1, becoming the 8-bit adder's external Cout. That lines up with 11111111 + 00000001 = 100000000 in binary (a 9-bit result: Cout=1, Sum=00000000), i.e. 255 + 1 = 256 in decimal — confirming that the ripple-carry chain behaves correctly across all eight stages.

Verilog Implementation Code

1-Bit full_adder module

```
module full_adder(
    input A,
    input B,
    input Cin,
    output Sum,
    output Cout
);

    assign Sum = A ^ B ^ Cin;
    assign Cout = (A & B) | (B & Cin) | (A & Cin);

endmodule
```

8-Bit full_adder_8bit module

```
module full_adder_8bit(
    input A0, A1, A2, A3, A4, A5, A6, A7,
    input B0, B1, B2, B3, B4, B5, B6, B7,
    input Cin,
    output S0, S1, S2, S3, S4, S5, S6, S7,
    output Cout
);

    wire c1, c2, c3, c4, c5, c6, c7;

    full_adder FA0 (.A(A0), .B(B0), .Cin(Cin), .Sum(S0), .Cout(c1));
    full_adder FA1 (.A(A1), .B(B1), .Cin(c1), .Sum(S1), .Cout(c2));
    full_adder FA2 (.A(A2), .B(B2), .Cin(c2), .Sum(S2), .Cout(c3));
    full_adder FA3 (.A(A3), .B(B3), .Cin(c3), .Sum(S3), .Cout(c4));
    full_adder FA4 (.A(A4), .B(B4), .Cin(c4), .Sum(S4), .Cout(c5));
    full_adder FA5 (.A(A5), .B(B5), .Cin(c5), .Sum(S5), .Cout(c6));
    full_adder FA6 (.A(A6), .B(B6), .Cin(c6), .Sum(S6), .Cout(c7));
    full_adder FA7 (.A(A7), .B(B7), .Cin(c7), .Sum(S7), .Cout(Cout));

endmodule
```

Verilog Testbench Code

1. Fixed-vector testbench (15 static test cases)

This testbench drives 15 chosen 8-bit input combinations — covering the all-zero case, the all-one case, single-bit cases, and several carry-overflow cases — into `full_adder_8bit`, holding each combination steady for 100 ns.

```
`timescale 1ns / 1ps

module full_adder_8bit_tb;

    reg A0, A1, A2, A3, A4, A5, A6, A7;
    reg B0, B1, B2, B3, B4, B5, B6, B7;
    reg Cin;
    wire S0, S1, S2, S3, S4, S5, S6, S7;
    wire Cout;

    full_adder_8bit uut (
        .A0(A0), .A1(A1), .A2(A2), .A3(A3), .A4(A4), .A5(A5), .A6(A6), .A7(A7),
        .B0(B0), .B1(B1), .B2(B2), .B3(B3), .B4(B4), .B5(B5), .B6(B6), .B7(B7),
        .Cin(Cin),
        .S0(S0), .S1(S1), .S2(S2), .S3(S3), .S4(S4), .S5(S5), .S6(S6), .S7(S7),
        .Cout(Cout)
    );

    initial begin

        // Test 1: 00000000 + 00000000 + 0 = 00000000
        {A7,A6,A5,A4,A3,A2,A1,A0} = 8'b00000000;
        {B7,B6,B5,B4,B3,B2,B1,B0} = 8'b00000000;
        Cin = 0; #100;

        // Test 2: 00000001 + 00000001 + 0 = 00000010
        {A7,A6,A5,A4,A3,A2,A1,A0} = 8'b00000001;
        {B7,B6,B5,B4,B3,B2,B1,B0} = 8'b00000001;
        Cin = 0; #100;

        // Test 3: 00000101 + 00000011 + 0 = 00001000
        {A7,A6,A5,A4,A3,A2,A1,A0} = 8'b00000101;
        {B7,B6,B5,B4,B3,B2,B1,B0} = 8'b00000011;
        Cin = 0; #100;

        // Test 4: 00001111 + 00001010 + 0 = 00011001
        {A7,A6,A5,A4,A3,A2,A1,A0} = 8'b00001111;
        {B7,B6,B5,B4,B3,B2,B1,B0} = 8'b00001010;
        Cin = 0; #100;

        // Test 5: 11111111 + 00000000 + 0 = 11111111
        {A7,A6,A5,A4,A3,A2,A1,A0} = 8'b11111111;
        {B7,B6,B5,B4,B3,B2,B1,B0} = 8'b00000000;
        Cin = 0; #100;

        // Test 6: 11111111 + 00000001 + 0 = 00000000, Cout=1
        {A7,A6,A5,A4,A3,A2,A1,A0} = 8'b11111111;
        {B7,B6,B5,B4,B3,B2,B1,B0} = 8'b00000001;
        Cin = 0; #100;

        // Test 7: 11111111 + 11111111 + 0 = 11111110, Cout=1
        {A7,A6,A5,A4,A3,A2,A1,A0} = 8'b11111111;
        {B7,B6,B5,B4,B3,B2,B1,B0} = 8'b11111111;
        Cin = 0; #100;

        // Test 8: 00001010 + 00010100 + 1 = 00011111
        {A7,A6,A5,A4,A3,A2,A1,A0} = 8'b00001010;
        {B7,B6,B5,B4,B3,B2,B1,B0} = 8'b00010100;
```

```

Cin = 1; #100;

// Test 9: 11111111 + 11111111 + 1 = 11111111, Cout=1
{A7,A6,A5,A4,A3,A2,A1,A0} = 8'b11111111;
{B7,B6,B5,B4,B3,B2,B1,B0} = 8'b11111111;
Cin = 1; #100;

// Test 10: 01010101 + 00101010 + 0 = 01111111
{A7,A6,A5,A4,A3,A2,A1,A0} = 8'b01010101;
{B7,B6,B5,B4,B3,B2,B1,B0} = 8'b00101010;
Cin = 0; #100;

// Test 11: 00110011 + 00001100 + 1 = 01000000
{A7,A6,A5,A4,A3,A2,A1,A0} = 8'b00110011;
{B7,B6,B5,B4,B3,B2,B1,B0} = 8'b00001100;
Cin = 1; #100;

// Test 12: 10000000 + 10000000 + 0 = 00000000, Cout=1
{A7,A6,A5,A4,A3,A2,A1,A0} = 8'b10000000;
{B7,B6,B5,B4,B3,B2,B1,B0} = 8'b10000000;
Cin = 0; #100;

// Test 13: 01111111 + 00000001 + 0 = 10000000
{A7,A6,A5,A4,A3,A2,A1,A0} = 8'b01111111;
{B7,B6,B5,B4,B3,B2,B1,B0} = 8'b00000001;
Cin = 0; #100;

// Test 14: 10101010 + 01010101 + 0 = 11111111
{A7,A6,A5,A4,A3,A2,A1,A0} = 8'b10101010;
{B7,B6,B5,B4,B3,B2,B1,B0} = 8'b01010101;

Cin = 0; #100;

// Test 15: 00000000 + 00000000 + 1 = 00000001
{A7,A6,A5,A4,A3,A2,A1,A0} = 8'b00000000;
{B7,B6,B5,B4,B3,B2,B1,B0} = 8'b00000000;
Cin = 1; #100;

$finish;
end

endmodule

```

2. Self-checking, randomized testbench

A second testbench generates 15 random 8-bit values for A and B along with a random Cin on every iteration using \$random, works out the mathematically expected 9-bit result ($\{Cin?, A\} + \{Cin?, B\} + Cin$), compares it with the circuit's actual output $\{Cout, S7..S0\}$, and prints a PASS/FAIL verdict for every case — automating the check instead of relying on reading waveforms by eye.

```

`timescale 1ns / 1ps

module full_adder_8bit_v2_tb_dynamic;

    // Inputs
    reg A0, A1, A2, A3, A4, A5, A6, A7;
    reg B0, B1, B2, B3, B4, B5, B6, B7;
    reg Cin;

    // Outputs
    wire S0, S1, S2, S3, S4, S5, S6, S7;
    wire Cout;

```

```

// Temporary 8-bit values
reg [7:0] a_temp;
reg [7:0] b_temp;

// Expected and actual 9-bit results
reg [8:0] expected;
reg [8:0] actual;

integer i;
integer pass_count;
integer fail_count;

// Instantiate the 8-bit Full Adder
full_adder_8bit uut (
    .A0(A0), .A1(A1), .A2(A2), .A3(A3),
    .A4(A4), .A5(A5), .A6(A6), .A7(A7),

    .B0(B0), .B1(B1), .B2(B2), .B3(B3),
    .B4(B4), .B5(B5), .B6(B6), .B7(B7),

    .Cin(Cin),

    .S0(S0), .S1(S1), .S2(S2), .S3(S3),
    .S4(S4), .S5(S5), .S6(S6), .S7(S7),

    .Cout(Cout)
);

// Dynamic testing
initial begin

    pass_count = 0;
    fail_count = 0;

    // Generate 15 random test cases
    for (i = 0; i < 15; i = i + 1) begin

        // Generate random 8-bit inputs
        a_temp = $random;
        b_temp = $random;

        // Generate random Carry-in
        Cin = $random;

        // Apply random inputs to individual ports
        {A7,A6,A5,A4,A3,A2,A1,A0} = a_temp;
        {B7,B6,B5,B4,B3,B2,B1,B0} = b_temp;

        // Wait for circuit response
        #100;

        // Calculate expected result
        expected = {1'b0, a_temp}
            + {1'b0, b_temp}
            + Cin;

        // Collect actual circuit output
        actual = {Cout,S7,S6,S5,S4,S3,S2,S1,S0};

        // Compare expected and actual
        if (actual == expected) begin

            $display(

```

```

        "Test %0d: PASS | A=%0d B=%0d Cin=%b | Expected=%0d Actual=%0d",
        i + 1,
        a_temp,
        b_temp,
        Cin,
        expected,
        actual
    );

    pass_count = pass_count + 1;

end
else begin

    $display(
        "Test %0d: FAIL | A=%0d B=%0d Cin=%b | Expected=%0d Actual=%0d",
        i + 1,
        a_temp,
        b_temp,
        Cin,
        expected,
        actual
    );

    fail_count = fail_count + 1;

end

end

// Final summary
$display("-----");
$display("Total Tests = %0d", pass_count + fail_count);
$display("PASS = %0d", pass_count);
$display("FAIL = %0d", fail_count);
$display("-----");

$finish;

end

endmodule

```

Port Definition

The `full_adder_8bit` module was set up in Xilinx ISE's New Source Wizard with 17 individual scalar input ports (A0-A7, B0-B7, Cin) and 9 individual scalar output ports (S0-S7, Cout), rather than as 8-bit bus ports. That was a deliberate choice: declaring A, B and Sum as single 8-bit buses (input [7:0] A, for instance) would make Xilinx ISE's RTL Viewer automatically collapse the eight identical `full_adder` instances into one compact vectorized symbol for readability, which would hide the individual FA0-FA7 blocks. Declaring the ports as separate scalar signals avoids that automatic merging, so the schematic viewer draws all eight full-adder instances individually, keeping the ripple-carry structure visible and easy to check by eye.

New Source Wizard

Define Module
Specify ports for module.

Entity name:

Architecture name:

Port Name	Direction	Bus	MSB	LSB
A	in	☒	7	0
B	in	☒	7	0
Cin	in	☐		
Sum	out	☒	7	0
Cout	out	☐		
	in	☐		
	in	☐		
	in	☐		

Circuit Diagram / Schematic

Once the design was synthesized (Synthesize - XST), Xilinx ISE produced the RTL schematic for full_adder_8bit. At the top level the module shows up as a single black-box symbol with 17 scalar inputs and 9 scalar outputs, as pictured below.

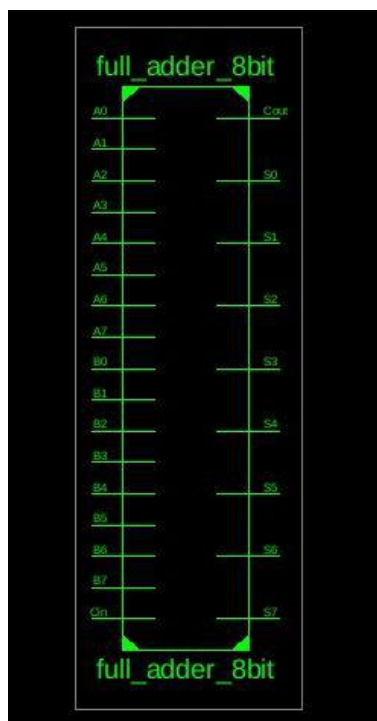


Fig 1: full_adder_8bit top-level black-box symbol

Opening this symbol reveals the internal structure: eight separate full_adder instances (FA0 through FA7), each linked to the next by its own internal carry wire (c1...c7), matching the ripple-carry chain from the structural Verilog exactly.

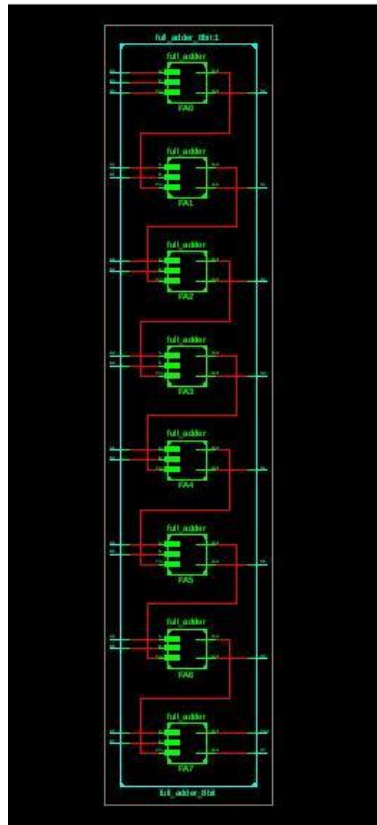


Fig 2: RTL schematic — FA0 through FA7 chained via the internal carry signals

The Hierarchy panel in ISE backs this up independently: it lists FA0 through FA7 as eight distinct sub-instances of full_adder sitting under full_adder_8bit, and the source-code pane alongside it shows exactly the structural instantiation used to build the chain.

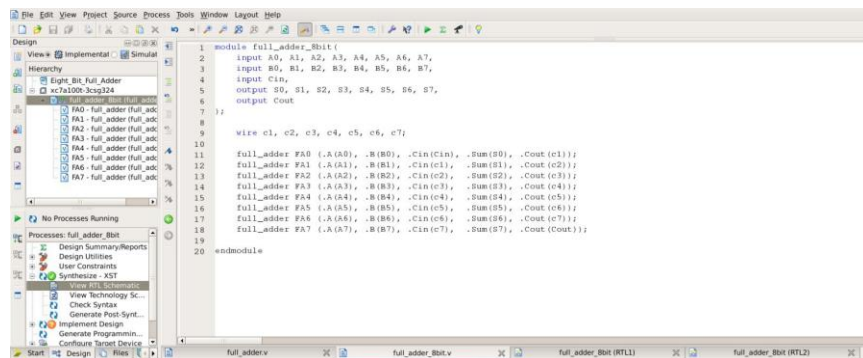


Fig 3: ISE Hierarchy panel showing FA0-FA7 as separate instances, alongside the source code

Expected Output — Test Vectors

Since an 8-bit adder has 2^{17} possible input combinations across A, B and Cin, an exhaustive truth table isn't practical here. Instead, 15 representative test vectors were selected and their expected Sum and Cout were worked out by hand using ordinary binary addition — spanning the all-zero case, the all-one case, single-bit and multi-bit additions, and several carry/overflow edge cases (Tests 6, 7, 9, 12 and 13 all exercise carry propagation across multiple stages of the ripple chain).

Quick-reference summary

Test	A (8-bit)	B (8-bit)	Cin	Expected Sum	Expected Cout
1	00000000	00000000	0	00000000	0
2	00000001	00000001	0	00000010	0
3	00000101	00000011	0	00001000	0
4	00001111	00001010	0	00011001	0
5	11111111	00000000	0	11111111	0
6	11111111	00000001	0	00000000	1
7	11111111	11111111	0	11111110	1
8	00001010	00010100	1	00011111	0
9	11111111	11111111	1	11111111	1
10	01010101	00101010	0	01111111	0
11	00110011	00001100	1	01000000	0
12	10000000	10000000	0	00000000	1
13	01111111	00000001	0	10000000	0
14	10101010	01010101	0	11111111	0
15	00000000	00000000	1	00000001	0

Detailed bit-by-bit (FA0 → FA7) calculation for every test case

For complete transparency, each test case below is worked through the same way the hardware itself computes it: stage by stage from FA0 (LSB) to FA7 (MSB), with every stage's Cout feeding directly into the next stage's Cin — reproducing the ripple-carry chain described above exactly.

Test 1: A = 00000000 B = 00000000 Cin = 0

Stage	FA0	FA1	FA2	FA3	FA4	FA5	FA6	FA7
A[i]	0	0	0	0	0	0	0	0
B[i]	0	0	0	0	0	0	0	0
Cin (in)	0	0	0	0	0	0	0	0
Sum[i]	0	0	0	0	0	0	0	0
Cout (out)	0	0	0	0	0	0	0	0

Result → Sum = 00000000 Cout = 0 (0 + 0 + 0 = 0)

Test 2: A = 00000001 B = 00000001 Cin = 0

Stage	FA0	FA1	FA2	FA3	FA4	FA5	FA6	FA7
A[i]	1	0	0	0	0	0	0	0
B[i]	1	0	0	0	0	0	0	0
Cin (in)	0	1	0	0	0	0	0	0
Sum[i]	0	1	0	0	0	0	0	0
Cout (out)	1	0	0	0	0	0	0	0

Result → Sum = 00000010 Cout = 0 (1 + 1 + 0 = 2)

Test 3: A = 00000101 B = 00000011 Cin = 0

Stage	FA0	FA1	FA2	FA3	FA4	FA5	FA6	FA7
A[i]	1	0	1	0	0	0	0	0
B[i]	1	1	0	0	0	0	0	0
Cin (in)	0	1	1	1	0	0	0	0
Sum[i]	0	0	0	1	0	0	0	0
Cout (out)	1	1	1	0	0	0	0	0

Result → Sum = 00001000 Cout = 0 (5 + 3 + 0 = 8)

Test 4: A = 00001111 B = 00001010 Cin = 0

Stage	FA0	FA1	FA2	FA3	FA4	FA5	FA6	FA7
A[i]	1	1	1	1	0	0	0	0
B[i]	0	1	0	1	0	0	0	0
Cin (in)	0	0	1	1	1	0	0	0
Sum[i]	1	0	0	1	1	0	0	0
Cout (out)	0	1	1	1	0	0	0	0

Result → Sum = 00011001 Cout = 0 (15 + 10 + 0 = 25)

Test 5: A = 11111111 B = 00000000 Cin = 0

Stage	FA0	FA1	FA2	FA3	FA4	FA5	FA6	FA7
A[i]	1	1	1	1	1	1	1	1
B[i]	0	0	0	0	0	0	0	0
Cin (in)	0	0	0	0	0	0	0	0
Sum[i]	1	1	1	1	1	1	1	1
Cout (out)	0	0	0	0	0	0	0	0

Result → Sum = 11111111 Cout = 0 (255 + 0 + 0 = 255)

Test 6: A = 11111111 B = 00000001 Cin = 0

Stage	FA0	FA1	FA2	FA3	FA4	FA5	FA6	FA7
A[i]	1	1	1	1	1	1	1	1
B[i]	1	0	0	0	0	0	0	0
Cin (in)	0	1	1	1	1	1	1	1
Sum[i]	0	0	0	0	0	0	0	0
Cout (out)	1	1	1	1	1	1	1	1

Result → Sum = 00000000 Cout = 1 (255 + 1 + 0 = 256)

Test 7: A = 11111111 B = 11111111 Cin = 0

Stage	FA0	FA1	FA2	FA3	FA4	FA5	FA6	FA7
A[i]	1	1	1	1	1	1	1	1
B[i]	1	1	1	1	1	1	1	1
Cin (in)	0	1	1	1	1	1	1	1

Sum[i]	0	1	1	1	1	1	1	1
Cout (out)	1	1	1	1	1	1	1	1

Result → Sum = 11111110 Cout = 1 (255 + 255 + 0 = 510)

Test 8: A = 00001010 B = 00010100 Cin = 1

Stage	FA0	FA1	FA2	FA3	FA4	FA5	FA6	FA7
A[i]	0	1	0	1	0	0	0	0
B[i]	0	0	1	0	1	0	0	0
Cin (in)	1	0	0	0	0	0	0	0
Sum[i]	1	1	1	1	1	0	0	0
Cout (out)	0	0	0	0	0	0	0	0

Result → Sum = 00011111 Cout = 0 (10 + 20 + 1 = 31)

Test 9: A = 11111111 B = 11111111 Cin = 1

Stage	FA0	FA1	FA2	FA3	FA4	FA5	FA6	FA7
A[i]	1	1	1	1	1	1	1	1
B[i]	1	1	1	1	1	1	1	1
Cin (in)	1	1	1	1	1	1	1	1
Sum[i]	1	1	1	1	1	1	1	1
Cout (out)	1	1	1	1	1	1	1	1

Result → Sum = 11111111 Cout = 1 (255 + 255 + 1 = 511)

Test 10: A = 01010101 B = 00101010 Cin = 0

Stage	FA0	FA1	FA2	FA3	FA4	FA5	FA6	FA7
A[i]	1	0	1	0	1	0	1	0
B[i]	0	1	0	1	0	1	0	0
Cin (in)	0	0	0	0	0	0	0	0
Sum[i]	1	1	1	1	1	1	1	0
Cout (out)	0	0	0	0	0	0	0	0

Result → Sum = 01111111 Cout = 0 (85 + 42 + 0 = 127)

Test 11: A = 00110011 B = 00001100 Cin = 1

Stage	FA0	FA1	FA2	FA3	FA4	FA5	FA6	FA7
A[i]	1	1	0	0	1	1	0	0
B[i]	0	0	1	1	0	0	0	0
Cin (in)	1	1	1	1	1	1	1	0
Sum[i]	0	0	0	0	0	0	1	0
Cout (out)	1	1	1	1	1	1	0	0

Result → Sum = 01000000 Cout = 0 (51 + 12 + 1 = 64)

Test 12: A = 10000000 B = 10000000 Cin = 0

Stage	FA0	FA1	FA2	FA3	FA4	FA5	FA6	FA7
A[i]	0	0	0	0	0	0	0	1
B[i]	0	0	0	0	0	0	0	1
Cin (in)	0	0	0	0	0	0	0	0
Sum[i]	0	0	0	0	0	0	0	0
Cout (out)	0	0	0	0	0	0	0	1

Result → Sum = 00000000 Cout = 1 (128 + 128 + 0 = 256)

Test 13: A = 01111111 B = 00000001 Cin = 0

Stage	FA0	FA1	FA2	FA3	FA4	FA5	FA6	FA7
A[i]	1	1	1	1	1	1	1	0
B[i]	1	0	0	0	0	0	0	0
Cin (in)	0	1	1	1	1	1	1	1
Sum[i]	0	0	0	0	0	0	0	1
Cout (out)	1	1	1	1	1	1	1	0

Result → Sum = 10000000 Cout = 0 (127 + 1 + 0 = 128)

Test 14: A = 10101010 B = 01010101 Cin = 0

Stage	FA0	FA1	FA2	FA3	FA4	FA5	FA6	FA7
A[i]	0	1	0	1	0	1	0	1
B[i]	1	0	1	0	1	0	1	0
Cin (in)	0	0	0	0	0	0	0	0
Sum[i]	1	1	1	1	1	1	1	1
Cout (out)	0	0	0	0	0	0	0	0

Result → Sum = 11111111 Cout = 0 (170 + 85 + 0 = 255)

Test 15: A = 00000000 B = 00000000 Cin = 1

Stage	FA0	FA1	FA2	FA3	FA4	FA5	FA6	FA7
A[i]	0	0	0	0	0	0	0	0
B[i]	0	0	0	0	0	0	0	0
Cin (in)	1	0	0	0	0	0	0	0
Sum[i]	1	0	0	0	0	0	0	0
Cout (out)	0	0	0	0	0	0	0	0

Result → Sum = 00000001 Cout = 0 (0 + 0 + 1 = 1)

Simulation and Result

The 8-bit full adder was first run through ISim using the fixed-vector testbench for 1500 ns, stepping through all 15 test cases (100 ns apiece) in the same order as the table above. The waveform below shows the A, B, Cin, S0-S7 and Cout signals across the whole run.



Fig 4: ISim waveform — fixed 15-vector testbench (full_adder_8bit_tb)

Reading the Sum(7:0) and Cout values off the waveform at each 100 ns mark and checking them against the values calculated above shows an exact match across all 15 test cases, with no stray unknown (X/U) states appearing anywhere.

The design was then re-checked independently with the self-checking randomized testbench, which produced 15 fresh random 8-bit combinations of A, B and Cin on each run and compared the circuit's output against an arithmetically computed expected value on the fly.

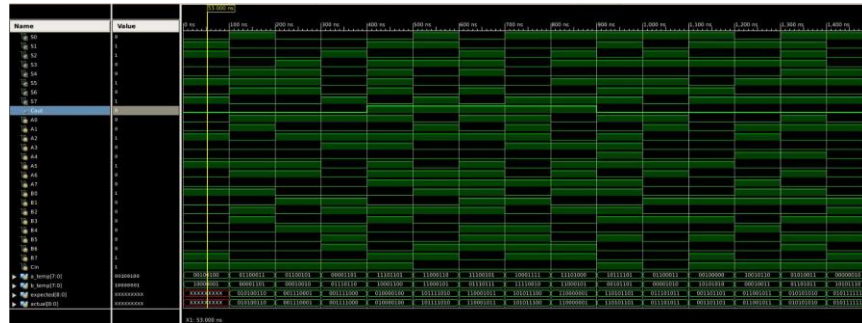


Fig 5: ISim waveform — self-checking randomized testbench, with the expected [8:0] and actual [8:0] tracks shown for direct comparison

As Fig. 5 shows, the expected [8:0] and actual [8:0] tracks read identically at every settled time step (apart from the initial X state before the first 100 ns delay finishes) — exactly the PASS condition the testbench checks for with `if (actual == expected)` — confirming correct ripple-carry behavior across randomized inputs, and not merely the 15 hand-picked cases.

Result Analysis

- The structural Verilog implementation (eight full adder instances chained via c1 to c7) correctly realizes an 8-bit ripple-carry adder — every one of the 15 fixed test vectors matched its hand-calculated expected Sum and Cout exactly.
- The randomized, self-checking testbench adds further confidence beyond the 15 fixed vectors, since it exercises different random combinations on each run and automatically flags any mismatch.
- Carry-overflow edge cases (Tests 6, 7, 9, 12, 13) — where the ripple carry has to propagate through several stages, or where the sum of two 8-bit numbers exceeds 8 bits — were all handled correctly, producing the right 8-bit Sum along with a correctly asserted Cout.
- Declaring scalar ports instead of bus ports let the RTL schematic show all eight full adder instances individually (Fig. 2), giving a clear, checkable view of the ripple-carry wiring that a bus-merged view would otherwise have hidden.
- This experiment confirms that the 1-bit full adder verified in Experiment 02 can be reliably reused as a building block for constructing wider, functionally correct arithmetic circuits.

GitHub Repository

[The completed source code](#)

Conclusion

This experiment took the verified 1-bit full adder from Experiment 02 and extended it into an 8-bit ripple-carry adder through structural instantiation in Verilog HDL. It covered the theory behind ripple-carry addition, structural (gate-level instantiation) Verilog coding, generating and reading the RTL schematic in Xilinx ISE, building both a fixed-vector and a self-checking randomized testbench, and running a behavioral simulation with ISim. The simulated Sum and Cout outputs matched the manually derived expected values for all 15 fixed test vectors, and the self-checking testbench further confirmed correct behavior across 15 randomized input combinations with zero mismatches — meeting every objective of the experiment and confirming that eight instances of one verified full-adder block, wired in a ripple-carry chain, correctly perform 8-bit binary addition.